A Quadratic Programming Based Teleoperation Scheme and Its Applications for Robotic Haircutting

***Abstract*—Teleoperation keeps a human operator in the loop and is a practical way to realize robotic haircutting, whereas heterogeneous master–slave manipulators and dedicated input devices hinder its application. In this article, a quadratic programming based teleoperation scheme is proposed, in which the slave manipulator follows the desired pose given by the operator within its joint-angle and joint-velocity limits, and no joint correspondence between the master and slave manipulators is needed. Then, the quadratic programming problem is solved online by a discrete-time projection neural network. It is proven that this network converges globally and linearly to the optimal solution, and that its output satisfies the joint limits at every time instant even if only one iteration is executed. Furthermore, the proposed scheme is applied to robotic haircutting in two applications. In the first application, the teleoperation with heterogeneous master–slave manipulators is realized on an SO101 master arm and a Piper slave arm with a mannequin head. In the second application, a smartphone-based operation mode via a visual Tag is realized, in which the smartphone replaces the dedicated master arm and provides a low-cost solution for robotic haircutting.**

***Index Terms*—Discrete-time projection neural network, heterogeneous teleoperation, quadratic programming (QP), robotic haircutting.**

I. Introduction

ROBOTIC haircutting is an emerging personal-care application of robot manipulators, which is motivated by the growing demand for accessible grooming assistance [1], [2]. Different from conventional industrial tasks, haircutting requires a cutting tool to move around the head in the presence of deformable hair, restricted visual access, and sensitive body regions. In recent years, several aspects of robotic haircutting have been investigated, e.g., camera-assisted self-haircutting [3], soft and compliant hair manipulation [4], CNC-inspired task decomposition [5], localization and coverage planning from a mobile robotics perspective [6], and layered safety strategies [7]. Nevertheless, fully autonomous haircutting remains difficult because hair modeling, perception, and personalized styling are still open problems [1], [2]. In view of this, teleoperation that keeps a human operator in the loop is a practical way to bring robotic haircutting into real applications [8].

Telerobotic systems, as a typical form of human–machine interaction, have received wide attention [9]. A variety of telerobotic control schemes have been developed, e.g., neural-learning-based control with collision avoidance [10], shared control for learning and generalizing skills [11], fixed-time neural learning control for flexible telerobots [12], finite-time composite learning control under networked time-varying delays [13], and passivity-based stabilization for physical human–(tele)robot interaction [14]. In addition, network transmission protocols [15], live remote interaction platforms [16], and cloud-based high-availability architectures [17] have been developed to support remote operation. Note that most of the above schemes focus on the dynamics, stability, or communication of master–slave systems. If the master and slave manipulators are heterogeneous, i.e., with different joint arrangements and workspaces, then the operator's command cannot be mapped joint by joint, and it has to be realized by the slave subject to its own kinematics and joint limits. Moreover, a dedicated master arm increases the equipment cost of remote operation, which limits the accessibility of telerobotic services.

Quadratic programming (QP) provides a unified way to handle task objectives and physical constraints of robot manipulators. For example, manipulability optimization [18], repetitive motion planning [19], and repetitive motion planning with obstacle avoidance at the position level [20] have been formulated as constrained QP problems. Efficient numerical solvers, e.g., ProxQP [21] and ElastiQP [22], are available for real-time robot control. Meanwhile, recurrent neural networks (RNNs) are widely adopted as QP solvers because of their parallel processing capability. A variety of neural solvers have been developed, e.g., penalty-based varying-parameter RNNs [23], adaptive variable-parameter dynamic learning networks [24], gradient-based differential neural networks [25], gradient-based integral neural dynamics [26], reciprocal of exponential varying-parameter RNNs [27], penalty-enhanced error-redefinition RNNs [28], and noise-suppressing zeroing neural networks under joint-angle constraints [29]. Furthermore, discrete-time neural solvers are compatible with digital controllers, and they have been developed via direct discretization [30], left-and-right two-step discretization [31], and five-instant discretization for the real-time control of a Stewart platform [32]. However, these QP-based schemes are mainly applied to the tracking of predefined trajectories, rather than to human-in-the-loop teleoperation.

In view of the above discussions, QP-based teleoperation is of great importance for robotic haircutting, since one optimization problem can take both the intent of the operator and the joint limits of the slave manipulator into account while the tool moves near the head. Although TeleHairing has recently provided a teleoperation baseline for robotic haircutting with smartphone AprilTag input [8], to the best of our knowledge, a QP-based teleoperation scheme for robotic haircutting with heterogeneous manipulators has not been reported. In this article, a QP-based teleoperation scheme is proposed, and its applications for robotic haircutting are presented. Specifically, a QP with joint-position and joint-velocity bounds converts the desired task-space pose given by the operator into joint commands of the slave, and this QP is solved online by a discrete-time projection neural network. The main contributions are listed below.

1) A QP-based teleoperation scheme for robotic haircutting is proposed, which is the first of its kind as far as we know, and its QP problem is solved online by a discrete-time projection neural network.
2) The proposed scheme realizes the teleoperation of robotic haircutting with heterogeneous master–slave manipulators, i.e., an SO101 master and a Piper slave, without requiring joint correspondence.
3) The proposed scheme realizes a smartphone-based operation mode via a visual Tag, which replaces the dedicated master arm and provides a low-cost solution for robotic haircutting.

The rest of this article is arranged as follows. Section II gives the preliminaries. Section III describes the optimization problem of haircutting teleoperation and designs a discrete-time projection neural network, whose convergence is analyzed in Section IV. Section V reports the experiments, and Section VI concludes this article.

II. Preliminaries

A. Notations

Throughout this article, scalars are written in lowercase letters (e.g., a, h), vectors in bold lowercase letters (e.g., $\mathbf{u}$, $\mathbf{q}$), and matrices in bold uppercase letters (e.g., $\mathbf{H}$, $\mathbf{J}$). $\mathbb{R}^n$ denotes the n-dimensional real space. Superscript T denotes the transpose operator; $\mathbf{I}_n$ is the identity matrix of size n, and $\mathbf{0}_n$ is the zero vector in $\mathbb{R}^n$. Inequalities, $\max\{\cdot\}$, and $\min\{\cdot\}$ for vectors are taken elementwise.

B. Projection Neural Network

For the optimization problem $\min_{\mathbf{u}\in\Omega} F(\mathbf{u})$, the projection neural network in [33] evolves as

$$\epsilon\frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} = -\mathbf{u} + \Pi_{\Omega}\left(\mathbf{u} - \gamma\nabla F(\mathbf{u})\right) \tag{1}$$

where $\epsilon > 0$ is a time constant; $\gamma > 0$ is a scaling parameter; $F : \mathbb{R}^n \to \mathbb{R}$ is the convex objective; $\Omega = \{\mathbf{u}|\boldsymbol{\ell} \le \mathbf{u} \le \boldsymbol{b}\} \subset \mathbb{R}^n$ is a box set defined by the bounds $\boldsymbol{\ell}$ and $\boldsymbol{b}$; and $\Pi_{\Omega}(\mathbf{x}) = \arg\min_{\mathbf{y}\in\Omega}\|\mathbf{x}-\mathbf{y}\| = \max\{\min\{\mathbf{x},\boldsymbol{b}\},\boldsymbol{\ell}\}$ is the projection onto Ω. The state of (1) is proven to be stable at the optimal solution of this problem [33].

III. Main Results

A. Problem Description

Consider a slave manipulator with n_s joints. At the kth time instant, the tracking error is defined as follows:

$$\mathbf{e}_k = \mathbf{r}_{d,k} - \mathbf{r}(\mathbf{q}_{s,k}) \tag{2}$$

where $\mathbf{q}_{s,k} \in \mathbb{R}^{n_s}$ is the vector of slave joint angles; the forward kinematics $\mathbf{r} : \mathbb{R}^{n_s} \to \mathbb{R}^6$ gives the slave pose, i.e., three position and three local orientation coordinates; and $\mathbf{r}_{d,k} \in \mathbb{R}^6$ is the desired pose.

In this work, the desired pose $\mathbf{r}_{d,k}$ has six dimensions, and the slave has six joints, i.e., $n_s = 6$, which means that the slave has no redundancy for tracking $\mathbf{r}_{d,k}$. In this case, accurate tracking of $\mathbf{r}_{d,k}$ and small joint velocities of the slave, which are both required in the teleoperation scheme, conflict with each other, especially if the slave is near singular configurations or the joint limits. As such, a trade-off between them is made, and the joint velocity $\mathbf{u}_k \in \mathbb{R}^{n_s}$ is computed from the following optimization problem, which respects the joint limits of the slave:

$$\begin{aligned}\min_{\mathbf{u}_k} \quad & \tfrac{1}{2}\left\|\mathbf{W}(\mathbf{J}_k\mathbf{u}_k - k_e\mathbf{e}_k)\right\|^2 + \tfrac{\lambda}{2}\left\|\mathbf{u}_k\right\|^2 \\ \text{s.t.} \quad & \boldsymbol{\ell}_k \le \mathbf{u}_k \le \boldsymbol{b}_k\end{aligned} \tag{3}$$

where $\mathbf{J}_k = \partial\mathbf{r}(\mathbf{q}_{s,k})/\partial\mathbf{q}_{s,k} \in \mathbb{R}^{6\times n_s}$ is the Jacobian matrix; $\mathbf{W} = \mathrm{diag}(w_p, w_p, w_p, w_r, w_r, w_r)$ is the weighting matrix with $w_p > 0$ and $w_r \ge 0$; $k_e > 0$ is the feedback gain; $\lambda > 0$ is the regularization coefficient. In Problem (3), the first term serves to track $\mathbf{r}_{d,k}$, i.e., the end-effector velocity $\mathbf{J}_k\mathbf{u}_k$ is driven toward $k_e\mathbf{e}_k$ so that $\mathbf{e}_k$ is reduced; the second term penalizes large joint velocities, which avoids abrupt motion of the slave near singular configurations. The trade-off is adjusted by λ, i.e., if λ is larger, then the motion of the slave is smoother, whereas the tracking error is larger. The lower and upper bounds of $\mathbf{u}_k$, i.e., $\boldsymbol{\ell}_k$ and $\boldsymbol{b}_k$, are defined as follows:

$$\boldsymbol{\ell}_k = \max\{-\mathbf{v}_{\max}, (\mathbf{q}_{\min} - \mathbf{q}_{s,k})/h_k\} \tag{4}$$

$$\boldsymbol{b}_k = \min\{\mathbf{v}_{\max}, (\mathbf{q}_{\max} - \mathbf{q}_{s,k})/h_k\} \tag{5}$$

where $\mathbf{q}_{\min}$ and $\mathbf{q}_{\max}$ bound the joint angles from below and above; $\mathbf{v}_{\max} > \mathbf{0}_{n_s}$ bounds the joint velocity; and $h_k > 0$ is the sampling interval. By omitting the constant term, Problem (3) is rewritten as the following QP problem:

$$\begin{aligned}\min_{\mathbf{u}_k} \quad & F_k(\mathbf{u}_k) = \tfrac{1}{2}\mathbf{u}_k^T\mathbf{H}_k\mathbf{u}_k + \mathbf{f}_k^T\mathbf{u}_k \\ \text{s.t.} \quad & \mathbf{u}_k \in \Omega_k\end{aligned} \tag{6}$$

where $\mathbf{H}_k = \mathbf{J}_k^T\mathbf{W}^2\mathbf{J}_k + \lambda\mathbf{I}_{n_s}$, $\mathbf{f}_k = -k_e\mathbf{J}_k^T\mathbf{W}^2\mathbf{e}_k$, and $\Omega_k = \{\mathbf{u}\,|\,\boldsymbol{\ell}_k \le \mathbf{u} \le \boldsymbol{b}_k\}$ is the feasible set of Problem (6). The gradient of $F_k(\mathbf{u})$ is as follows:

$$\nabla F_k(\mathbf{u}) = \mathbf{H}_k\mathbf{u} + \mathbf{f}_k. \tag{7}$$

B. Design of a Discrete-Time Projection Neural Network

A discrete-time projection neural network for solving Problem (6) is designed by three steps.

Step 1: By [33], for any $\gamma > 0$, a vector $\mathbf{u}$ solves Problem (6) if and only if it satisfies

$$\mathbf{u} = \Pi_{\Omega_k}\left(\mathbf{u} - \gamma\nabla F_k(\mathbf{u})\right). \tag{8}$$

Step 2: In analogy with projection neural network (1) with $\Omega = \Omega_k$ and $F = F_k$, the following continuous-time model is obtained:

$$\epsilon\frac{\mathrm{d}\mathbf{u}}{\mathrm{d}t} = -\mathbf{u} + \Pi_{\Omega_k}\left(\mathbf{u} - \gamma\nabla F_k(\mathbf{u})\right). \tag{9}$$

The equilibrium of model (9) is exactly the solution of (8), and thus it is the optimal solution to Problem (6).

Step 3: Since the slave manipulator is controlled by a digital controller, model (9) is discretized by the Euler forward method with time step $\tau > 0$, and one step is executed at each time instant. Let $\alpha = \tau/\epsilon$.

Algorithm 1 Discrete-Time Projection Neural Network at the kth Time Instant

Require: $\mathbf{q}_{s,k}$, $\mathbf{r}_{d,k}$, $\mathbf{u}_{k-1}$ ($\mathbf{u}_{-1}=\mathbf{0}_{n_s}$); parameters $\mathbf{W}$, k_e, λ, α, γ.

Ensure: $\mathbf{u}_k$ and $\mathbf{q}_{\mathrm{cmd},k}$.

1: Compute $\mathbf{e}_k$, $\mathbf{J}_k$, $\mathbf{H}_k$, $\mathbf{f}_k$, $\boldsymbol{\ell}_k$, and $\boldsymbol{b}_k$;
2: Project $\mathbf{u}_{k-1}$ onto Ω_k, i.e., $\mathbf{u}_{k-1} \leftarrow \Pi_{\Omega_k}(\mathbf{u}_{k-1})$;
3: Set $\mathbf{u}_k = \mathbf{u}_{k-1}+\alpha[-\mathbf{u}_{k-1}+\Pi_{\Omega_k}(\mathbf{u}_{k-1}-\gamma\nabla F_k(\mathbf{u}_{k-1}))]$;
4: Set $\mathbf{q}_{\mathrm{cmd},k} = \mathbf{q}_{s,k} + h_k\mathbf{u}_k$.

Combining the three steps, the discrete-time projection neural network for Problem (6) reads

$$\mathbf{u}_k = \mathbf{u}_{k-1}+\alpha\left[-\mathbf{u}_{k-1}+\Pi_{\Omega_k}\left(\mathbf{u}_{k-1}-\gamma\nabla F_k(\mathbf{u}_{k-1})\right)\right] \quad (10)$$

where $\mathbf{u}_k \in \mathbb{R}^{n_s}$ is the output neuronal vector at the kth time instant, $\mathbf{u}_{k-1}$ is the output at the $(k-1)$th time instant, which is projected onto Ω_k before (10) is executed, i.e., $\mathbf{u}_{k-1} \in \Omega_k$, with $\mathbf{u}_{-1} = \mathbf{0}_{n_s}$, $\alpha \in (0,1]$ is the step size, and $\gamma > 0$ is the scaling parameter. In discrete-time projection neural network (10), the gradient term $\gamma\nabla F_k(\mathbf{u}_{k-1})$ reduces the tracking error, and the projection $\Pi_{\Omega_k}(\cdot)$ enforces the joint-angle and joint-velocity limits.

The step size α is adjustable. Note that (10) is equivalent to $\mathbf{u}_k = (1-\alpha)\mathbf{u}_{k-1}+\alpha\Pi_{\Omega_k}(\mathbf{u}_{k-1}-\gamma\nabla F_k(\mathbf{u}_{k-1}))$, which acts as a first-order low-pass filter whose inertia is $1-\alpha$. If α is smaller, then the joint-velocity command varies more smoothly between consecutive time instants, whereas $\mathbf{u}_k$ approaches the optimal solution $\mathbf{u}_k^\star$ of Problem (6) more slowly.

The joint-angle command sent to the slave is

$$\mathbf{q}_{\mathrm{cmd},k} = \mathbf{q}_{s,k} + h_k\mathbf{u}_k. \quad (11)$$

The computation at one time instant is summarized in Algorithm 1.

IV. Convergence Analysis

This section proves that the equilibrium of discrete-time projection neural network (10) is optimal and that the network converges. At the kth time instant, $\mathbf{q}_{s,k}$, $\mathbf{r}_{d,k}$, $\mathbf{H}_k$, and $\mathbf{f}_k$ are fixed, and the slave joint angles satisfy the joint limits, i.e., $\mathbf{q}_{\min} \le \mathbf{q}_{s,k} \le \mathbf{q}_{\max}$. Then, (4) and (5) yield $\boldsymbol{\ell}_k \le \mathbf{0}_{n_s} \le \boldsymbol{b}_k$, which implies that Ω_k is nonempty. For convenience of analysis, let

$$\mathcal{T}_k(\mathbf{u}) = \Pi_{\Omega_k}\left(\mathbf{u}-\gamma\nabla F_k(\mathbf{u})\right). \quad (12)$$

Then, discrete-time projection neural network (10) is rewritten as $\mathbf{u}_k = (1-\alpha)\mathbf{u}_{k-1} + \alpha\mathcal{T}_k(\mathbf{u}_{k-1})$, and its equilibrium $\bar{\mathbf{u}}_k$ satisfies $\bar{\mathbf{u}}_k = \mathcal{T}_k(\bar{\mathbf{u}}_k)$, i.e., if $\mathbf{u}_{k-1} = \bar{\mathbf{u}}_k$, then $\mathbf{u}_k = \bar{\mathbf{u}}_k$.

Theorem 1: Problem (6) has a unique optimal solution $\mathbf{u}_k^\star$, and $\bar{\mathbf{u}}_k = \mathbf{u}_k^\star$.

Proof: The equilibrium $\bar{\mathbf{u}}_k$ satisfies the following equation:

$$\bar{\mathbf{u}}_k = \Pi_{\Omega_k}\left(\bar{\mathbf{u}}_k-\gamma\nabla F_k(\bar{\mathbf{u}}_k)\right). \quad (13)$$

Equation (13) coincides with (8) for $\mathbf{u} = \bar{\mathbf{u}}_k$, so $\bar{\mathbf{u}}_k$ solves Problem (6) by Step 1.

For any nonzero $\mathbf{z} \in \mathbb{R}^{n_s}$, the following inequality holds:

$$\mathbf{z}^T\mathbf{H}_k\mathbf{z} = \left\|\mathbf{W}\mathbf{J}_k\mathbf{z}\right\|^2 + \lambda\left\|\mathbf{z}\right\|^2 \ge \lambda\left\|\mathbf{z}\right\|^2 > 0. \quad (14)$$

It implies that $\mathbf{H}_k$ is positive definite and $F_k(\mathbf{u})$ is strictly convex. Since Ω_k is nonempty, closed, bounded, and convex, Problem (6) has a unique optimal solution $\mathbf{u}_k^\star$. As such, $\bar{\mathbf{u}}_k = \mathbf{u}_k^\star$. Conversely, $\mathbf{u}_k^\star$ satisfies (8) according to Step 1. Hence, $\mathbf{u}_k^\star$ is the unique equilibrium of discrete-time projection neural network (10), which completes the proof. ∎

Lemma 1 (See [33]): $\left\|\Pi_{\Omega_k}(\mathbf{y}) - \Pi_{\Omega_k}(\mathbf{x})\right\| \le \left\|\mathbf{y}-\mathbf{x}\right\|$ $\forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^{n_s}$.

Denote by σ_k and $\bar{\sigma}_k$ the smallest and largest eigenvalues of $\mathbf{H}_k$.

Theorem 2: If $0 < \alpha \le 1$ and $0 < \gamma < 2/\bar{\sigma}_k$, then the output of discrete-time projection neural network (10) satisfies $\mathbf{u}_k \in \Omega_k$ and

$$\left\|\mathbf{u}_k - \mathbf{u}_k^\star\right\| \le \rho_k\left\|\mathbf{u}_{k-1} - \mathbf{u}_k^\star\right\| \quad (15)$$

where $\rho_k = 1-\alpha(1-c_k) \in [0,1)$ and $c_k = \max\{|1-\gamma\sigma_k|, |1-\gamma\bar{\sigma}_k|\}$. Moreover, if Problem (6) remains unchanged for $k \ge k_0$, i.e., $\mathbf{H}_k = \mathbf{H}$, $\mathbf{f}_k = \mathbf{f}$, and $\Omega_k = \Omega$, then the output of (10) converges globally to the optimal solution $\mathbf{u}^\star$ at the linear rate ρ.

Proof: According to Theorem 1, $\mathbf{u}_k^\star = \mathcal{T}_k(\mathbf{u}_k^\star)$. Then, (10) yields

$$\mathbf{u}_k - \mathbf{u}_k^\star = (1-\alpha)(\mathbf{u}_{k-1} - \mathbf{u}_k^\star) + \alpha\left[\mathcal{T}_k(\mathbf{u}_{k-1}) - \mathcal{T}_k(\mathbf{u}_k^\star)\right]. \quad (16)$$

Based on Lemma 1 and (12), one has

$$\left\|\mathcal{T}_k(\mathbf{x}) - \mathcal{T}_k(\mathbf{y})\right\| \le \left\|\mathbf{I}_{n_s} - \gamma\mathbf{H}_k\right\|_2\left\|\mathbf{x}-\mathbf{y}\right\| \quad \forall \mathbf{x}, \mathbf{y} \in \mathbb{R}^{n_s} \quad (17)$$

with $\|\cdot\|_2$ denoting the matrix spectral norm. Since the eigenvalues of $\mathbf{H}_k$ belong to $[\sigma_k, \bar{\sigma}_k]$, $\left\|\mathbf{I}_{n_s} - \gamma\mathbf{H}_k\right\|_2 = c_k$. If $0 < \gamma < 2/\bar{\sigma}_k$, then $0 \le c_k < 1$, and thus $\rho_k \in [0,1)$ for $0 < \alpha \le 1$. Since $1-\alpha \ge 0$, (16) and (17) yield

$$\begin{aligned}\left\|\mathbf{u}_k - \mathbf{u}_k^\star\right\| &\le \left[(1-\alpha)+\alpha c_k\right]\left\|\mathbf{u}_{k-1} - \mathbf{u}_k^\star\right\| \\ &= \rho_k\left\|\mathbf{u}_{k-1} - \mathbf{u}_k^\star\right\|.\end{aligned}$$

Since $\mathbf{u}_{k-1} \in \Omega_k$, $\mathcal{T}_k(\mathbf{u}_{k-1}) \in \Omega_k$, and Ω_k is convex, $\mathbf{u}_k = (1-\alpha)\mathbf{u}_{k-1} + \alpha\mathcal{T}_k(\mathbf{u}_{k-1}) \in \Omega_k$.

Now, Problem (6) is assumed to remain unchanged for $k \ge k_0$. Since $\mathbf{u}_k \in \Omega$, the projection of $\mathbf{u}_k$ onto Ω is $\mathbf{u}_k$ itself for $k \ge k_0$. Consider the Lyapunov function $V_k = \left\|\mathbf{u}_{k-1} - \mathbf{u}^\star\right\|^2$. According to (15), the difference of V_k is derived as follows:

$$\Delta V_k = V_{k+1} - V_k \le \rho^2 V_k - V_k = -(1-\rho^2)V_k. \quad (18)$$

Since $1-\rho^2 > 0$, $\Delta V_k \le 0$, and $\Delta V_k = 0$ implies $V_k = 0$, i.e., $\mathbf{u}_{k-1} = \mathbf{u}^\star$. Hence, $\mathbf{u}^\star$ is a globally asymptotically stable equilibrium of (10). Moreover, (18) leads to $V_{k+1} \le \rho^2 V_k$, and thus $\left\|\mathbf{u}_k - \mathbf{u}^\star\right\| \le \rho^{k-k_0+1}\left\|\mathbf{u}_{k_0-1} - \mathbf{u}^\star\right\|$ by induction, which completes the proof. ∎

Theorem 3: Let the conditions of Theorem 2 hold. At every time instant, whether or not $\mathbf{u}_k$ has reached $\mathbf{u}_k^\star$, the output $\mathbf{u}_k$ of (10) and the command (11) satisfy

$$\left\|\mathbf{u}_k - \mathbf{u}_k^\star\right\| \le \frac{\sqrt{n_s}}{1-c_k}\, r_k(\mathbf{u}_k) \quad (19)$$

$$-\mathbf{v}_{\max} \le \mathbf{u}_k \le \mathbf{v}_{\max} \quad (20)$$

$$\mathbf{q}_{\min} \le \mathbf{q}_{\mathrm{cmd},k} \le \mathbf{q}_{\max} \quad (21)$$

where $r_k(\mathbf{u}) = \left\|\mathbf{u} - \mathcal{T}_k(\mathbf{u})\right\|_\infty$ is the residual.

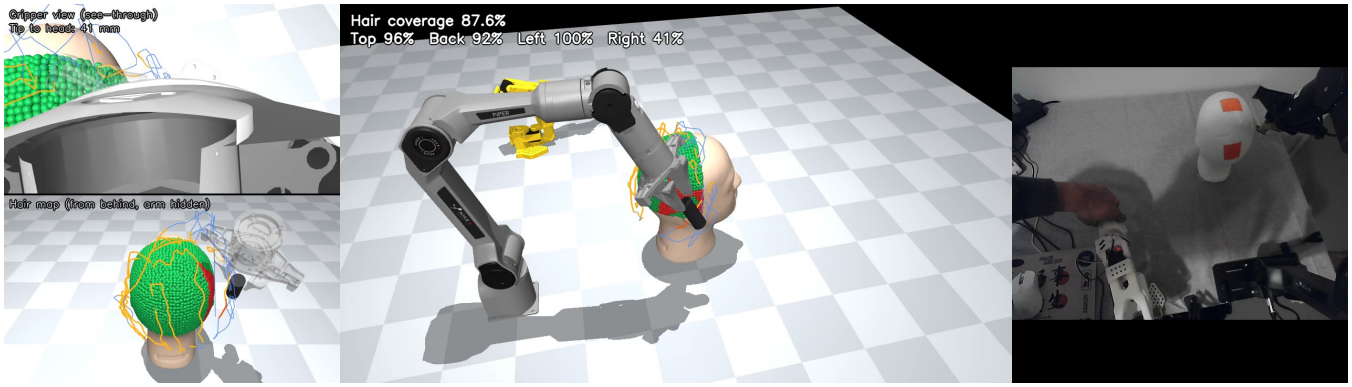
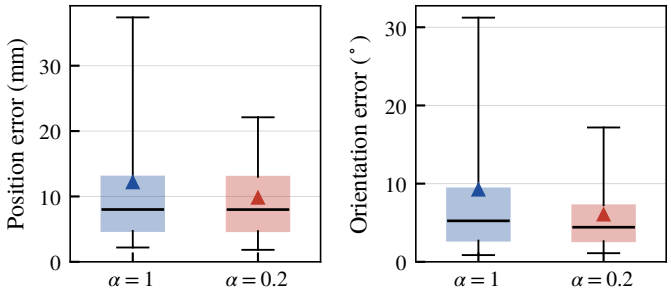


Fig. 1. Snapshot of the master-arm mode with $\alpha = 1$, including the simulation views (left and center) and the camera view (right).



(a) (b)

Fig. 2. Box plots of the feedback tracking errors in the master-arm mode, where the boxes, lines, whiskers, and triangles denote the quartiles, medians, 5th–95th percentiles, and means, respectively. (a) Position error. (b) Orientation error.

Proof: Since $\mathbf{u}_k^\star = \mathcal{T}_k(\mathbf{u}_k^\star)$, the triangle inequality and (17) yield

$$\|\mathbf{u}_k - \mathbf{u}_k^\star\| \le \|\mathbf{u}_k - \mathcal{T}_k(\mathbf{u}_k)\| + c_k \|\mathbf{u}_k - \mathbf{u}_k^\star\| .$$

Rearranging the above inequality and using $\|\mathbf{x}\| \le \sqrt{n_s}\,\|\mathbf{x}\|_\infty$ yield (19).

According to Theorem 2, $\mathbf{u}_k \in \Omega_k$. In view of (4) and (5), $\mathbf{u}_k$ satisfies (20) and

$$(\mathbf{q}_{\min} - \mathbf{q}_{s,k})/h_k \le \mathbf{u}_k \le (\mathbf{q}_{\max} - \mathbf{q}_{s,k})/h_k.$$

Multiplying the above inequality by $h_k > 0$ and substituting it into (11) yield (21). ■

V. Experiment Results

The proposed QP-based teleoperation scheme and its two haircutting applications are evaluated in this section with respect to tracking accuracy, the effect of joint limits, computational efficiency, and task coverage.

A. Experimental Setup

A six-joint Piper arm is used as the slave, and a tool mounted on the slave is guided around a mannequin head. The two applications are tested, i.e., the master-arm mode and the smartphone mode. In both modes, a MuJoCo visualization displays the robot motion and the model-based coverage.

TABLE I
Tracking Errors and Computation Time in the Master-Arm Mode

Quantity	α	Median	Mean	RMS	Max.
Position error (mm)	1	8.00	12.12	20.76	183.82
	0.2	7.99	9.76	12.32	69.21
Orientation error (°)	1	5.24	9.20	16.31	98.14
	0.2	4.43	6.01	8.14	45.29
Computation time (ms)	1	0.72	0.72	0.74	2.48
	0.2	0.73	0.73	0.75	2.23

The desired, command-derived, and feedback-derived poses, the commanded and measured joint angles, and the computation time are recorded at each time instant.

Let $\mathbf{p}_{d,i} \in \mathbb{R}^3$, $\mathbf{p}_{c,i} \in \mathbb{R}^3$, and $\mathbf{p}_{r,i} \in \mathbb{R}^3$ denote the desired, command-derived, and feedback-derived flange positions of the ith sample ($i \in \{1, \ldots, N\}$), respectively, which are expressed in meters in the base frame of the slave. The position errors in millimeters are defined as follows:

$$e_{p,i} = 1000\,\|\mathbf{p}_{r,i} - \mathbf{p}_{d,i}\|\,, \tag{22}$$

$$e_{p,c,i} = 1000\,\|\mathbf{p}_{c,i} - \mathbf{p}_{d,i}\|\,. \tag{23}$$

The orientation errors are defined as the magnitudes of the corresponding relative rotation vectors in degrees. Note that the command-derived error results from the feedback correction, regularization, and joint limits, which is different from the error bounded in (19).

B. Heterogeneous Master–Slave Teleoperation

In this experiment, as shown in Fig. 1, an SO101 arm is used as the master, and the Piper arm is used as the slave. The end-effector pose of the master is used as the desired pose of the slave. Two trials with $\alpha = 1$ and $\alpha = 0.2$ are conducted. The other parameters are $w_p = 1$, $w_r = 0.1$, $k_e = 30$, $\lambda = 0.001$, and $\gamma = 1/\bar{\sigma}_k$, the joint-velocity limit is 90°/s, and the control frequency is 100 Hz. The joint limits are $[-150, 150]$, $[15, 150]$, $[-160, -15]$, $[-100, 100]$, $[-65, 65]$, and $[-100, 100]$ degrees for joints 1–6. The two trials last 234.57 s and 290.83 s, respectively. Since the operator moves the master freely, the two trials are compared in terms of statistics.

Table I and Fig. 2 summarize the tracking errors, and Figs. 3 and 4 show that the slave follows the desired position and

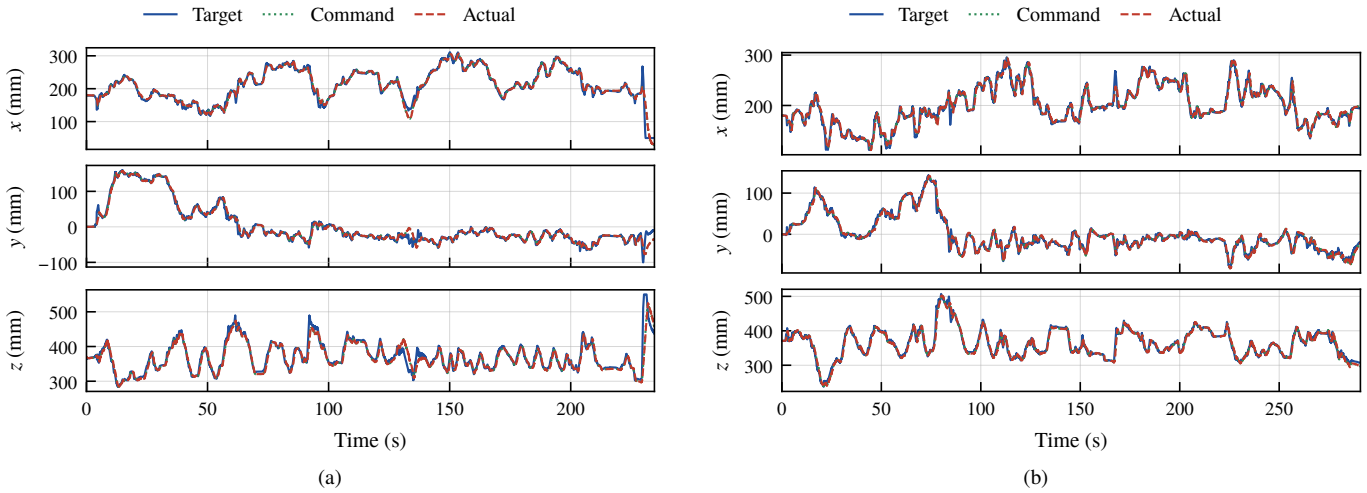


Fig. 3. Cartesian position tracking in the master-arm mode, where Target, Command, and Actual denote the desired, command-derived, and feedback-derived positions, respectively. (a) $\alpha = 1$. (b) $\alpha = 0.2$.

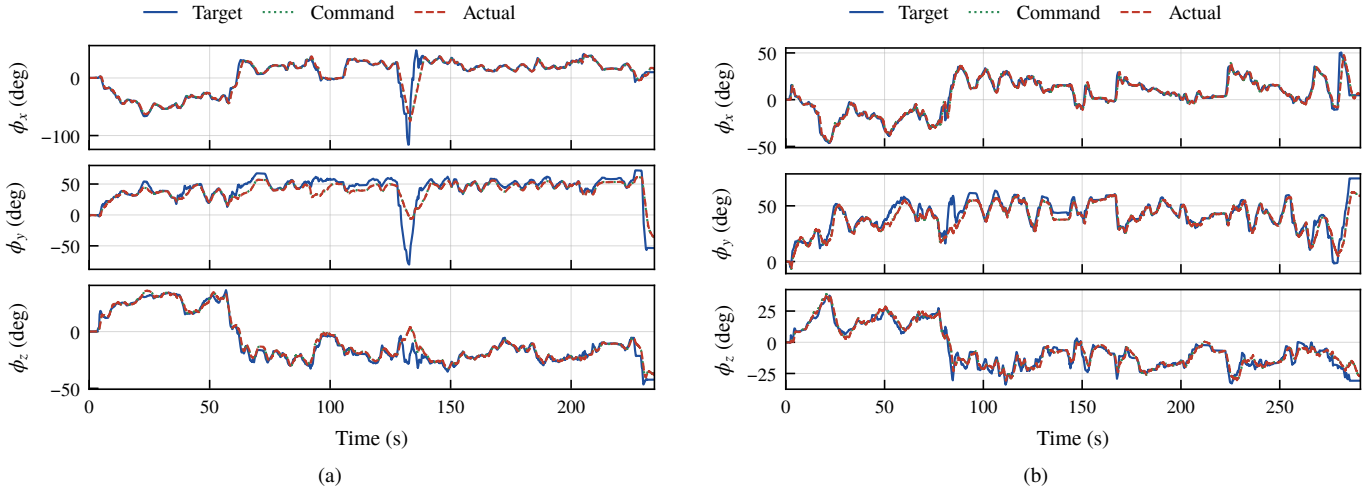


Fig. 4. Orientation tracking in the master-arm mode, where ϕ_x, ϕ_y, and ϕ_z denote the components of the rotation vector relative to the initial desired orientation. (a) $\alpha = 1$. (b) $\alpha = 0.2$.

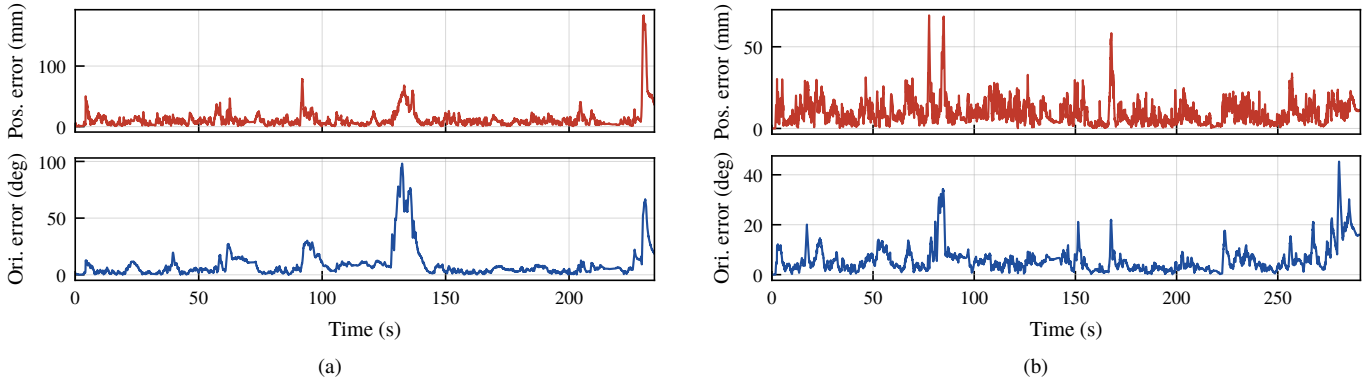


Fig. 5. Feedback position and orientation errors in the master-arm mode. (a) $\alpha = 1$. (b) $\alpha = 0.2$.

orientation in both trials. Compared with $\alpha = 1$, $\alpha = 0.2$ leads to smaller mean position and orientation errors, i.e., 9.76 mm and 6.01° versus 12.12 mm and 9.20°, whereas the median position errors are almost the same, i.e., 7.99 mm and 8.00 mm. The larger errors for $\alpha = 1$ mainly result from two segments in Figs. 4(a) and 5(a). From 129 s to 141 s, joint 5 approaches zero, as shown in Fig. 6(a), and the slave is close to a wrist singularity, where the actual orientation cannot follow the rapid change of the desired orientation. In the last 5 s, the operator lifts the master, and the desired pose leaves the reachable region of the slave.

Fig. 4 shows that the orientation error mainly occurs in ϕ_y.

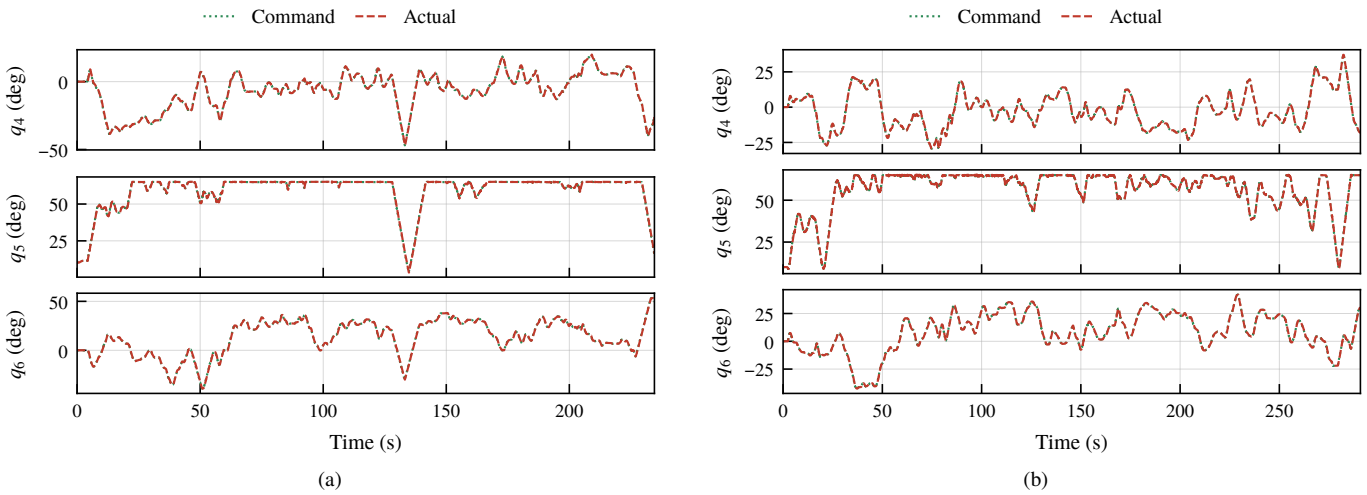


Fig. 6. Commanded and measured angles of joints 4–6 in the master-arm mode. (a) $\alpha = 1$. (b) $\alpha = 0.2$.

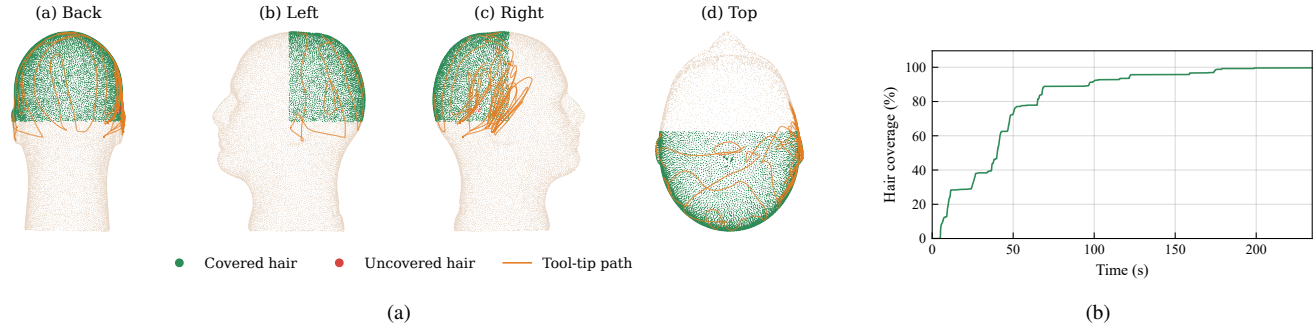


Fig. 7. Coverage of the modeled hair region in the master-arm mode with $\alpha = 1$. (a) Projection of the tool-tip path onto the head surface from four views, where the green and red points denote the covered and uncovered hair, respectively. (b) Cumulative coverage.

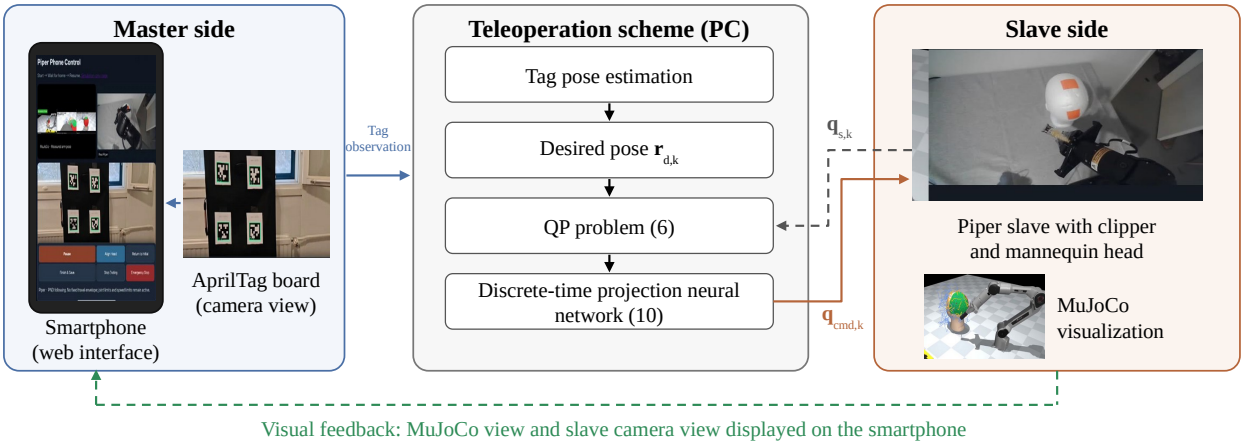


Fig. 8. Schematic of the smartphone-based teleoperation via Tag.

For $\alpha = 1$, the actual ϕ_y remains smaller than the desired one from 65 s to 125 s, where joint 5 stays at its upper limit, as shown in Fig. 6(a). If the desired orientation requires joint 5 to exceed its limit, then the orientation cannot be fully achieved, whereas the position is still tracked, since the weight of the orientation error w_r is smaller than that of the position error w_p. Specifically, for $\alpha = 1$, the median orientation error is 6.06° when joint 5 is at its upper limit, whereas it is 3.41° otherwise.

As shown in Fig. 6 and Table II, joint 5 frequently stays at its upper limit, where a sample is labeled as near-limit if any commanded joint angle is within 0.5° of its limit. All commanded joint angles remain within the joint limits in both trials, which verifies Theorem 3. Table II shows that, for $\alpha = 1$, the mean errors away from the limits are larger than those near the limits, since all samples in the above two segments are away from the limits. For $\alpha = 0.2$, the mean position errors near and away from the limits are close, i.e., 9.34 mm and

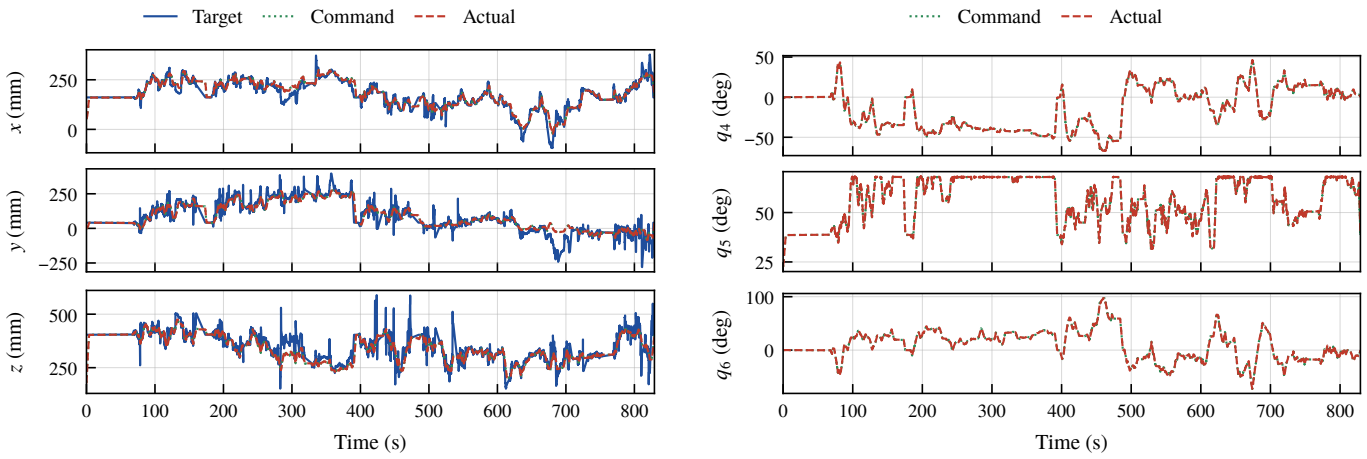


Fig. 9. Tracking in the smartphone mode. (a) Desired (Target), command-derived (Command), and feedback-derived (Actual) positions. (b) Commanded and measured angles of joints 4–6.

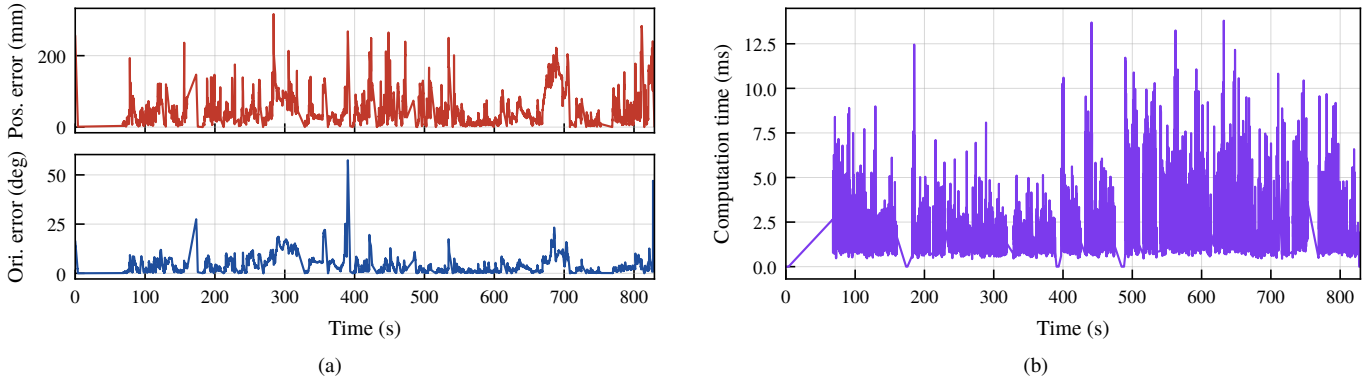


Fig. 10. Tracking errors and computation time in the smartphone mode. (a) Feedback position and orientation errors. (b) Computation time at each time instant.

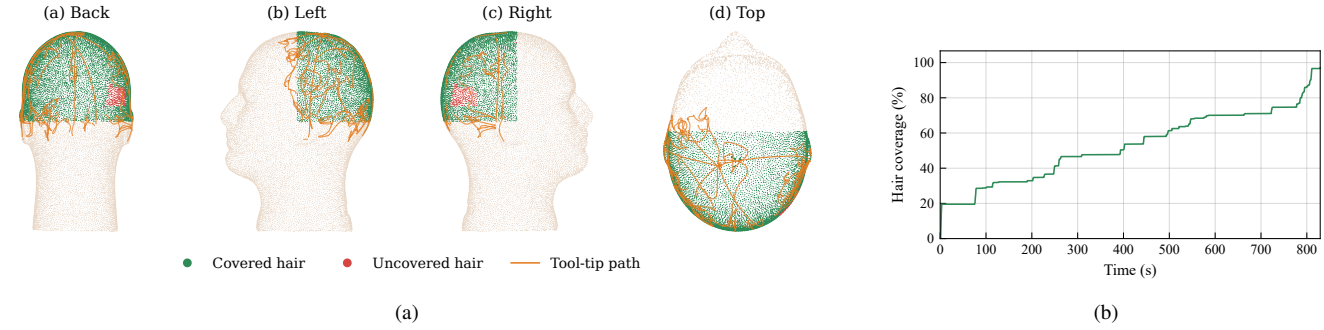


Fig. 11. Coverage of the modeled hair region in the smartphone mode. (a) Projection of the tool-tip path onto the head surface from four views, where the green and red points denote the covered and uncovered hair, respectively. (b) Cumulative coverage.

TABLE II
MEAN TRACKING ERRORS GROUPED BY COMMANDED LIMIT PROXIMITY

Mode	Group	Samples	Pos. (mm)	Ori. (°)
Master arm, $\alpha = 1$	Away	8260	17.70	12.45
	Near	14095	8.86	7.29
Master arm, $\alpha = 0.2$	Away	19197	9.95	5.61
	Near	8515	9.34	6.91
Smartphone	Away	19368	42.73	3.06
	Near	11785	61.06	6.80

9.95 mm, whereas the mean orientation error near the limits is larger, i.e., 6.91° versus 5.61°, which is consistent with the above orientation analysis.

The mean computation time is about 0.7 ms in both trials, which is much shorter than the sampling interval of 10 ms. Fig. 7 shows that the geometric coverage, i.e., the area of the head model within 30 mm of the tool tip, finally reaches 99.7% for $\alpha = 1$, and it is 99.5% for $\alpha = 0.2$. These results show that robotic haircutting can be teleoperated by the proposed scheme with heterogeneous master–slave manipulators. Moreover, $\alpha = 0.2$ leads to a smoother command and smaller peak errors than

TABLE III
COMPARISON BETWEEN THE TWO OPERATION MODES

Quantity	Master arm	Smartphone
Mean position error (mm)	12.12	49.67
Mean orientation error (°)	9.20	4.47
Mean computation time (ms)	0.72	1.88
Geometric coverage (%)	99.7	96.96

$\alpha = 1$, which is more suitable for haircutting near the head.

C. Smartphone-Based Teleoperation via Tag

In this experiment, the master arm is replaced by a smartphone. As shown in Fig. 8, the camera of the smartphone observes a board with four AprilTags, and the estimated pose is used as the desired pose of the slave. The smartphone also displays the MuJoCo visualization and the camera view of the slave. In this experiment, (10) is repeated at each time instant until convergence. The experiment lasts 829.2 s, with a median sampling interval of 24 ms.

Fig. 9(a) shows that the desired position supplied by the hand-held smartphone contains rapid variations, whereas the slave follows a smoother trajectory owing to the joint-velocity limits, which leads to the short peaks of the position error in Fig. 10(a). Table III shows that both modes complete the haircutting task. The master-arm mode reaches a coverage of 99.7% with a mean computation time of 0.72 ms, and the smartphone mode reaches 96.96% with 1.88 ms.

Fig. 9(b) and Table II show that joint 5 often reaches its upper limit, and that the samples near the limits have larger tracking errors than the other samples. The commanded angle of joint 5 exceeds its upper limit by at most 0.11°, only when the measured angle has already exceeded the limit due to the overshoot of the slave, i.e., $\mathbf{q}_{s,k} \leq \mathbf{q}_{\max}$ does not hold, and the command drives joint 5 back toward its limit.

Fig. 10(b) shows that the computation time remains shorter than the sampling interval, with a mean of 1.88 ms. Fig. 11 shows that the geometric coverage finally reaches 96.96%. Hence, the smartphone can take the place of the dedicated master arm, and the same scheme teleoperates robotic haircutting with a low-cost device.

VI. CONCLUSION

In this article, a QP-based teleoperation scheme has been proposed, in which the slave manipulator follows the desired pose given by the operator within its joint-angle and joint-velocity limits, and no joint correspondence between the master and slave manipulators is needed. Then, the QP problem has been solved online by a discrete-time projection neural network. It has been proven that this network converges globally and linearly to the optimal solution, and that its output satisfies the joint limits at every time instant even if only one iteration is executed. Furthermore, the proposed scheme has been applied to robotic haircutting in two applications, i.e., the teleoperation with heterogeneous master–slave manipulators on an SO101–Piper system and a smartphone-based operation mode via a visual Tag, where the latter provides a low-cost solution for robotic haircutting. Further investigations may aim at the filtering of the desired pose supplied by the smartphone, the evaluation under remote communication, the reduction of the tracking error caused by joint limits, and the evaluation of physical haircutting outcomes.

REFERENCES

[1] S. Li, *Haircutting Robots*. Cham, Switzerland: Springer, 2025.

[2] A. T. Khan and S. Li, "Robotic haircutting systems: A survey of methods, challenges, and hair modeling insights," *IEEE J. Sel. Areas Sensors*, vol. 3, pp. 104–112, 2026.

[3] K. Futami, T. Terada, and M. Tsukamoto, "A system for supporting self-haircuts using camera equipped robot," in *Proc. Int. Conf. Adv. Mobile Comput. Multimedia*, Dec. 2014.

[4] U. Yoo, N. Dennler, E. Xing, M. Mataric, S. Nikolaidis, J. Ichnowski, and J. Oh, "Soft and compliant contact-rich hair manipulation and care," in *Proc. ACM/IEEE Int. Conf. Human-Robot Interact. (HRI)*, Mar. 2025, pp. 610–619.

[5] A. T. Khan and S. Li, "CNC-inspired robotic hair cutting: A comprehensive survey on precision personal care automation," *J. Artif. Intell. Autom.*, vol. 1, no. 1, Jan. 2026, Art. no. 2.

[6] Z. Huang, A. T. Khan, and S. Li, "Haircutting robots: A mobile robotics perspective," *Int. J. Syst. Sci.*, 2026.

[7] Z. Huang, A. Vilkki, S. Li, and J. Röning, "Safety in robotic haircutting," *J. Field Robot.*, 2026.

[8] Z. Huang, A. Vilkki, J. Huang, B. Liao, C. Luo, L. Angrisani, and S. Li, "TeleHairing: A teleoperation baseline for robotic haircutting," 2026, arXiv:2610.07096v1.

[9] G. Li, Q. Li, C. Yang, Y. Su, Z. Yuan, and X. Wu, "The classification and new trends of shared control strategies in telerobotic systems: A survey," *IEEE Trans. Haptics*, vol. 16, no. 2, pp. 118–133, Apr.–Jun. 2023.

[10] C. Yang, X. Wang, L. Cheng, and H. Ma, "Neural-learning-based telerobot control with guaranteed performance," *IEEE Trans. Cybern.*, vol. 47, no. 10, pp. 3148–3159, Oct. 2017.

[11] X. Shao and L. Xie, "A telerobotic shared control architecture for learning and generalizing skills in unstructured environments," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 15 953–15 965, 2025.

[12] Y. Yang, H. Jiang, L. Gan, C. Hua, and J. Li, "Fixed-time composite neural learning control of flexible telerobotic systems," *IEEE Trans. Cybern.*, vol. 54, no. 6, pp. 3602–3614, Jun. 2024.

[13] Y. Yang, H. Jiang, C. Hua, and J. Li, "Finite-time composite learning control for nonlinear teleoperation systems under networked time-varying delays," *Sci. China Inf. Sci.*, vol. 67, no. 6, Jun. 2024, Art. no. 162203.

[14] P. Paik, S. Thudi, and S. F. Atashzar, "Power-based velocity-domain variable structure passivity signature control for physical human-(tele)robot interaction," *IEEE Trans. Robot.*, vol. 39, no. 1, pp. 386–398, Feb. 2023.

[15] X. Liu, Y. Xue, Y. Wang, Z. Song, J. Ye, X. Ma, and N. Xi, "A new network transmission protocol for Internet-based telerobotic operations," *IEEE Internet Things J.*, vol. 12, no. 24, pp. 54 116–54 132, Dec. 2025.

[16] X. Wang, Y. Wu, J. Yu, Y. Wang, E. Liu, D. Zhao, and G. Liu, "TeleRobot: Design and implementation of a live remote interaction platform for robots," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2023, pp. 408–412.

[17] S. Gu, O. M. Chisom, J. Yang, and D. Yoon, "Cloud-based remote-controlled robot system: A Kubernetes-orchestrated approach to high availability," *Expert Syst. Appl.*, vol. 285, 2025, Art. no. 127860.

[18] L. Jin, S. Li, H. M. La, and X. Luo, "Manipulability optimization of redundant manipulators using dynamic neural networks," *IEEE Trans. Ind. Electron.*, vol. 64, no. 6, pp. 4710–4720, Jun. 2017.

[19] Z. Xie, L. Jin, X. Du, X. Xiao, H. Li, and S. Li, "On generalized RMP scheme for redundant robot manipulators aided with dynamic neural networks and nonconvex bound constraints," *IEEE Trans. Ind. Informat.*, vol. 15, no. 9, pp. 5172–5181, Sep. 2019.

[20] W. Li, Z. Yi, R. Rao, D. Guo, Y. Pan, and K. Huang, "A quadratic programming solution to position-level repetitive motion planning and obstacle avoidance of joint-constrained robots," *IEEE Trans. Ind. Electron.*, vol. 72, no. 2, pp. 1860–1870, Feb. 2025.

[21] A. Bambade, F. Schramm, S. El-Kazdadi, S. Caron, A. Taylor, and J. Carpentier, "ProxQP: An efficient and versatile quadratic programming solver for real-time robotics applications and beyond," *IEEE Trans. Robot.*, 2025.

[22] D. Morton and J. Arrizabalaga, "ElastiQP: An always-feasible QP solver for constrained robot control," 2026, arXiv:2609.19080v1.

[23] Z. Zhang, S. Yang, and L. Zheng, "A penalty strategy combined varying-parameter recurrent neural network for solving time-varying multi-type constrained quadratic programming problems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 7, pp. 2993–3004, Jul. 2021.

[24] Z. Zhang, X. Sun, X. Li, and Y. Liu, "An adaptive variable-parameter dynamic learning network for solving constrained time-varying QP problem," *Neural Netw.*, vol. 184, Apr. 2025, Art. no. 106968.

[25] B. Liao, Y. Zeng, T. Wang, and Z. Li, "Gradient-based differential neural network to time-varying constrained quadratic programming," *Expert Syst. Appl.*, vol. 264, 2025, Art. no. 125893.

[26] Z. Sun, C. Yu, J. Zhou, L. Jin, and G. Wang, "Gradient-based integral neural dynamics model for solving time-varying quadratic programming: Application to manipulator," *IEEE Trans. Ind. Informat.*, 2026.

[27] Y. Wang, Y. Han, Z. Tang, W. Liu, and P. Wang, "Reciprocal of exponential varying-parameter RNN solving repetitive tracking control problems with tolerance of random initial error compounded with noise perturbation," *IEEE Trans. Ind. Informat.*, vol. 19, no. 11, pp. 10 998–11 007, Nov. 2023.

[28] T. Zhang and S. Yang, "Penalty-enhanced error-redefinition RNN for time-varying QP problems with multiple constraints and robot arm applications," *Nonlinear Dyn.*, vol. 113, pp. 23 259–23 283, 2025.

[29] Z. Sun, Y. Fei, S. Tang, X. Xiao, J. Luo, and K. Liu, "A noise suppression zeroing neural network for trajectory tracking with joint angle constraints of mobile manipulator," *Eng. Appl. Artif. Intell.*, vol. 133, 2024, Art. no. 108173.

[30] Y. Shi, W. Su, X. Cao, M. Liu, J. Wang, and D. K. Gerontitis, "Novel RNN and its enhanced variant based on direct discretization approach for discrete-form time-dependent quadratic programming," *IEEE Trans. Autom. Sci. Eng.*, vol. 23, pp. 1399–1410, 2026.

[31] M. Yang, Z. Zhu, J. Li, P. Guo, H. Zhang, and S. Li, "Novel left-and-right two-step discrete ZNN model for time-dependent nonlinear optimization with equality and inequality constraints applied to robot manipulator," *IEEE Trans. Ind. Electron.*, 2026, early access.

[32] Y. Shi, W. Sheng, J. Wang, L. Jin, B. Li, and X. Sun, "Real-time tracking control and efficiency analyses for Stewart platform based on discrete-time recurrent neural network," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 8, pp. 5099–5111, Aug. 2024.

[33] Y. Xia, H. Leung, and J. Wang, "A projection neural network and its application to constrained optimization problems," *IEEE Trans. Circuits Syst. I, Fundam. Theory Appl.*, vol. 49, no. 4, pp. 447–458, Apr. 2002.